

AI Integration, Deployment & Scalable Backend Architecture

NANDINI BHIMINENI

MEKA HRISHI TEJA CHOWDARY

Overview

The Raritone backend is designed as a scalable, AI-ready, and realtime-enabled backend infrastructure for supporting:

- Ecommerce workflows
- Virtual try-on systems
- Avatar generation
- Recommendation engines
- Optimized media delivery

The backend is built using:

- Node.js
- Express.js
- MongoDB
- Socket.IO
- Cloudinary CDN

The architecture is prepared for future AI/ML integration and cloud deployment.

Complete Backend Flow Diagram

Mobile App / Frontend



Backend API



Socket.IO Realtime Layer



AI Services



Cloudinary CDN



MongoDB Atlas



Response Returned To Frontend

Backend ↔ AI Service Communication

The backend acts as the middle layer between frontend applications and AI services.

Backend responsibilities:

- Request validation
- Authentication
- Image upload handling
- Forwarding requests to AI services
- Realtime updates
- Database storage
- Optimized media delivery

Future AI Workflow

User Uploads Image



Backend Receives Request



Validation Middleware Executes



Sharp Compression Applied



Backend Sends Data To AI Service



AI Generates Try-On Result



Cloudinary Upload



MongoDB Stores Result



Realtime Socket.IO Update Sent



Frontend Receives Final Result

AI Features Supported

The backend architecture is prepared for:

- Virtual try-on processing
- Avatar generation
- Recommendation systems
- Body compatibility analysis
- Personalized styling systems

Realtime Socket.IO Flow

Socket.IO enables:

- Realtime try-on updates
- Live processing notifications
- Instant frontend communication

Workflow:

Frontend Sends Try-On Request



Backend Starts AI Processing



Realtime Status Events Sent



AI Result Generated



Frontend Receives Instant Update

Benefits:

- Reduced API polling
- Better user experience
- Faster realtime updates

Cloudinary Workflow

Cloudinary handles:

- Image storage
- Optimized delivery
- CDN acceleration
- Image transformations

Workflow:

Image Upload



Sharp Compression



Cloudinary Upload



Optimized CDN URL



Frontend Delivery

Benefits:

- Faster image loading
- Reduced bandwidth usage
- Scalable media infrastructure

MongoDB Workflow

MongoDB stores:

- Users
- Products

- Try-on history
- Measurements
- Avatar data
- Orders
- Wishlist data

Benefits:

- Scalable document storage
- Flexible schema design
- AI-ready data structure

Deployment Options Research

Render

Supports:

- Node.js backend deployment
- Environment variables
- GitHub integration
- Scalable services

Benefits:

- Simple deployment
- Beginner-friendly infrastructure

Railway

Supports:

- Backend deployment
- MongoDB integration
- Automatic deployments

Benefits:

- Easy infrastructure setup
- Fast deployment workflow

AWS

Useful services:

- EC2
- S3
- Lambda
- CloudFront

Benefits:

- Enterprise-level scalability
- AI infrastructure support
- Distributed systems support

MongoDB Atlas

Provides:

- Cloud-hosted MongoDB
- Distributed clusters
- Backups
- Monitoring

Benefits:

- Scalable database infrastructure
- High availability

Cloudinary CDN

Provides:

- Cloud media storage
- Automatic optimization
- CDN acceleration

Benefits:

- Optimized media delivery
- Scalable storage

Scaling Research

Redis Caching

Redis can support:

- API caching
- Session management
- Realtime scaling
- AI processing cache

Benefits:

- Faster API responses
- Reduced database load

Queue Systems

Useful for:

- AI processing queues
- Background jobs
- Async workflows

Examples:

- BullMQ
- RabbitMQ

Benefits:

- Prevents backend blocking
- Scalable AI processing

Microservices Architecture

Future services may be separated into:

- Authentication service
- AI service
- Media service
- Realtime service

Benefits:

- Independent scaling
- Easier maintenance

Load Balancing

Distributes traffic across multiple backend servers.

Benefits:

- Reduced overload
- Better uptime
- Scalable infrastructure

CDN Optimization

Cloudinary CDN optimization includes:

- Automatic compression
- Automatic modern formats
- Optimized image delivery

Benefits:

- Improved frontend performance
- Lower bandwidth usage

Database Scaling

MongoDB Atlas scaling strategies:

- Indexing
- Replica sets
- Sharding

Benefits:

- Scalable database performance
- Distributed storage

AI Server Requirements

Future AI infrastructure may require:

- GPU servers
- Distributed AI processing
- AI task queues
- Scalable cloud storage

Possible tools:

- Docker
- Kubernetes
- GPU cloud instances

Security Architecture

Implemented security includes:

- JWT authentication
- Joi validation
- Helmet security
- Rate limiting
- Centralized error middleware

Benefits:

- Protected APIs
- Secure user data
- Reliable backend communication

Progress Report

Completed Features

Implemented:

- Authentication APIs
- Product APIs
- Measurement APIs
- Try-on APIs
- Wishlist APIs
- Order APIs
- Socket.IO integration
- Cloudinary workflows
- Sharp compression
- Joi validation

- Helmet security
- Morgan logging
- Rate limiting
- Centralized error handling

AI Integration Preparation

Prepared:

- AI-ready APIs
- Avatar architecture
- ProductMapping schema
- Realtime workflow planning
- Backend ↔ AI communication structure

Future Improvements

Possible future enhancements:

- Redis implementation
- Distributed Socket.IO scaling
- GPU AI servers
- Recommendation engine
- AI queue systems
- Microservices deployment
- Realtime AI processing

Conclusion

The Raritone backend architecture provides a scalable, cloud-ready, and AI-ready infrastructure for future virtual try-on workflows, realtime communication, optimized media handling, and AI/ML integration.

The backend is prepared for:

- Scalable deployments
- Realtime user interactions
- AI service integration
- Cloud-based infrastructure

- Future recommendation systems